```
import os, random, zipfile, tarfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

tf.random.set_seed(42)
random.seed(42)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
GPU: []
```

```
# COLAB: download straight into Colab's disk
FILE_ID = "1tWV-0UHFYxj3NCURqwGDV4fQUxzXruA9"        # the part
between /d/ and /view in the share link
ARCHIVE = "/content/brain_data.zip"

!gdown {FILE_ID} -O {ARCHIVE}
print(f"Downloaded: {os.path.getsize(ARCHIVE) / 1e6:.1f} MB")     # a
few KB means the download failed

# JUPYTER (exam day): no download. Skip the two lines above and point
to the zip where it already is:
# ARCHIVE = "/home/student/Desktop/brain_data.zip"      # <-- full
path of the zip on that machine
# print(os.path.exists(ARCHIVE))                         # must print
True
#https://drive.google.com/file/d/1tWV-0UHFYxj3NCURqwGDV4fQUxzXruA9/
view?usp=drive_link
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1tWV-
0UHFYxj3NCURqwGDV4fQUxzXruA9
From (redirected): https://drive.google.com/uc?id=1tWV-
0UHFYxj3NCURqwGDV4fQUxzXruA9&confirm=t&uuid=c1b9eacc-8c06-402d-a4d3-
38428092fb99
To: /content/brain_data.zip
100% 91.0M/91.0M [00:01<00:00, 67.0MB/s]
Downloaded: 91.0 MB
```

```
# COLAB
DATA_DIR = "/content/data"
# JUPYTER (exam day): extract into a folder next to the notebook (a
```

```
relative path is fine)
# DATA_DIR = "data"

with zipfile.ZipFile(ARCHIVE) as z:
    z.extractall(DATA_DIR, members=[m for m in z.namelist() if not
m.startswith("__MACOSX")])

print(os.listdir(DATA_DIR))
# JUPYTER (exam day): the data is already extracted on the machine, so
skip everything above
# and set only this line to the full path of the extracted folder:
# DATA_DIR = "/home/student/Desktop/brain_tumor"      # <-- full path,
case-sensitive on Linux
# print(os.listdir(DATA_DIR))                         # you should see
the class or train/test folders
```

```
['Testing', 'Training']
```

```
IMG_EXT = (".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff")
DATA_ROOT = DATA_DIR      # same variable on Colab and on Jupyter

for root, dirs, files in os.walk(DATA_ROOT):
    dirs[:] = sorted(d for d in dirs if not d.startswith((".",
"__MACOSX")))
    depth = root[len(DATA_ROOT):].count(os.sep)
    imgs = sum(f.lower().endswith(IMG_EXT) for f in files)
    other = len(files) - imgs
    print("  " * depth + f"{os.path.basename(root) or DATA_ROOT}/  ->
{len(dirs)} subfolders, {imgs} images, {other} other files")
    if len(dirs) > 6:
        print("  " * (depth + 1) + f"(showing 3 of {len(dirs)}
subfolders: {dirs[:3]})")
        dirs[:] = dirs[:3]
```

```
data/  -> 2 subfolders, 0 images, 0 other files
  Testing/  -> 4 subfolders, 0 images, 0 other files
    glioma_tumor/  -> 0 subfolders, 100 images, 0 other files
    meningioma_tumor/  -> 0 subfolders, 115 images, 0 other files
    no_tumor/  -> 0 subfolders, 105 images, 0 other files
    pituitary_tumor/  -> 0 subfolders, 74 images, 0 other files
  Training/  -> 4 subfolders, 0 images, 0 other files
    glioma_tumor/  -> 0 subfolders, 826 images, 0 other files
    meningioma_tumor/  -> 0 subfolders, 822 images, 0 other files
    no_tumor/  -> 0 subfolders, 395 images, 0 other files
    pituitary_tumor/  -> 0 subfolders, 827 images, 0 other files
```

```
# COLAB and JUPYTER: same cell, DATA_DIR comes from Cell 2.
# JUPYTER (exam day): Linux is case-sensitive. Check
print(os.listdir(DATA_DIR)) and change
# "Training" / "Testing" below if the folders are named differently.
```

```
TRAIN_DIR = DATA_DIR + "/Training"
TEST_DIR = DATA_DIR + "/Testing"

TRAIN_GLIOMA = TRAIN_DIR + "/glioma_tumor"
TRAIN_MENINGIOMA = TRAIN_DIR + "/meningioma_tumor"
TRAIN_NOTUMOR = TRAIN_DIR + "/no_tumor"
TRAIN_PITUITARY = TRAIN_DIR + "/pituitary_tumor"

TEST_GLIOMA = TEST_DIR + "/glioma_tumor"
TEST_MENINGIOMA = TEST_DIR + "/meningioma_tumor"
TEST_NOTUMOR = TEST_DIR + "/no_tumor"
TEST_PITUITARY = TEST_DIR + "/pituitary_tumor"

n_glioma = len(os.listdir(TRAIN_GLIOMA))
n_meningioma = len(os.listdir(TRAIN_MENINGIOMA))
n_notumor = len(os.listdir(TRAIN_NOTUMOR))
n_pituitary = len(os.listdir(TRAIN_PITUITARY))

print("Train glioma:    ", n_glioma)         # expect 826
print("Train meningioma:", n_meningioma)     # expect 822
print("Train no_tumor:  ", n_notumor)        # expect 395
print("Train pituitary: ", n_pituitary)      # expect 827
print("Test glioma:    ", len(os.listdir(TEST_GLIOMA)))        #
expect 100
print("Test meningioma:", len(os.listdir(TEST_MENINGIOMA)))    #
expect 115
print("Test no_tumor:  ", len(os.listdir(TEST_NOTUMOR)))       #
expect 105
print("Test pituitary: ", len(os.listdir(TEST_PITUITARY)))     #
expect 74
```

```
Train glioma:     826
Train meningioma: 822
Train no_tumor:   395
Train pituitary:  827
Test glioma:     100
Test meningioma: 115
Test no_tumor:   105
Test pituitary:  74
```

```
random.seed(42)
paths = []
paths += [TRAIN_GLIOMA + "/" + f for f in
random.sample(os.listdir(TRAIN_GLIOMA), 50)]
paths += [TRAIN_MENINGIOMA + "/" + f for f in
random.sample(os.listdir(TRAIN_MENINGIOMA), 50)]
paths += [TRAIN_NOTUMOR + "/" + f for f in
random.sample(os.listdir(TRAIN_NOTUMOR), 50)]
paths += [TRAIN_PITUITARY + "/" + f for f in
random.sample(os.listdir(TRAIN_PITUITARY), 50)]
```

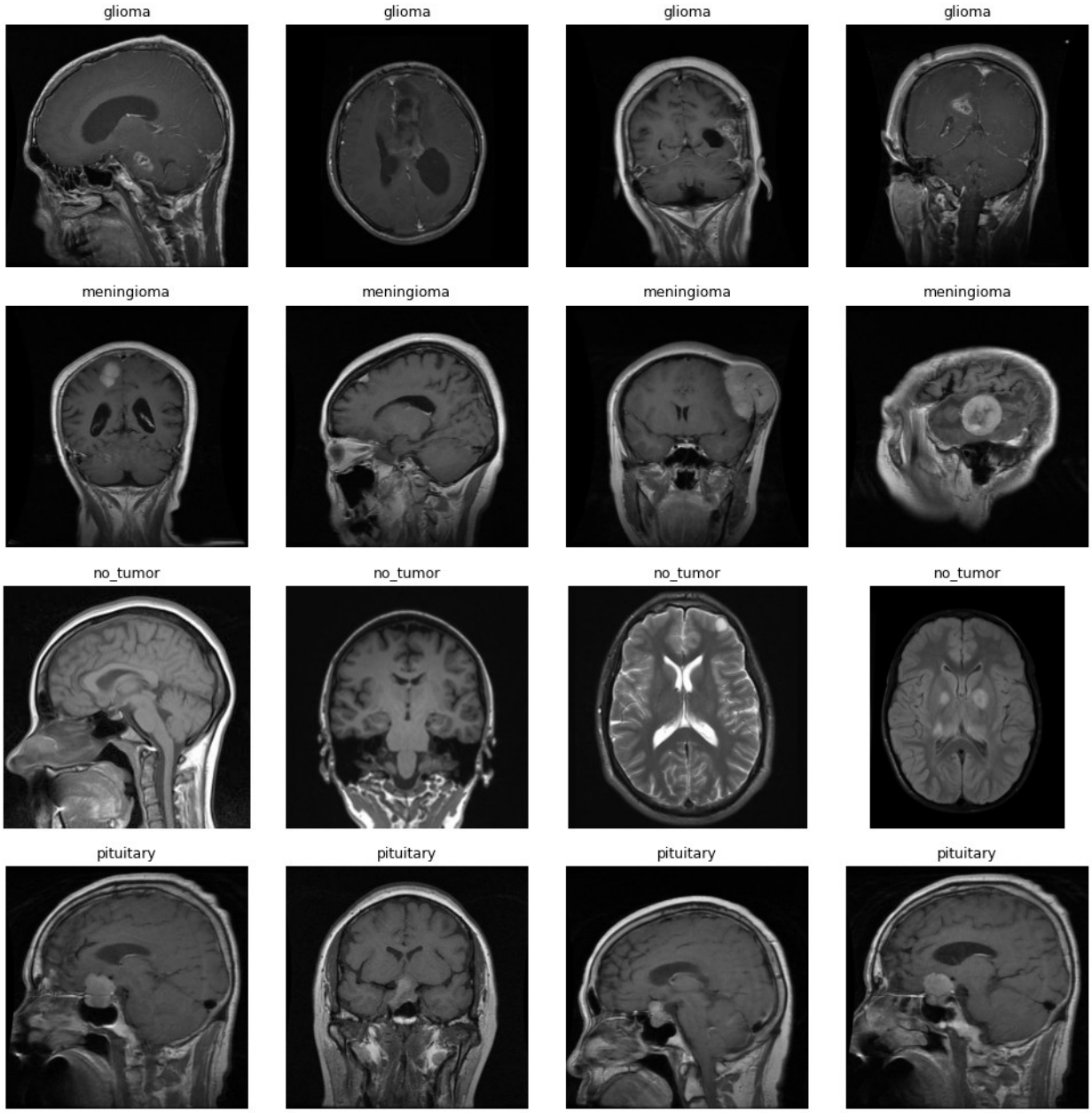
```
info = []
for p in paths:
    with Image.open(p) as im:
        info.append({"w": im.size[0], "h": im.size[1], "mode":
im.mode})
info = pd.DataFrame(info)

print(info[["w", "h"]].describe().loc[["min", "mean", "max"]])
print(info["mode"].value_counts())
```

```
             w        h
min    201.00   202.00
mean   465.82   466.58
max   1024.00  1075.00
mode
RGB    200
Name: count, dtype: int64
```

```
def show_row(folder, row, title):
    files = random.sample(os.listdir(folder), 4)
    for k in range(4):
        axes[row, k].imshow(Image.open(folder + "/" + files[k]),
cmap="gray")
        axes[row, k].set_title(title, fontsize=9)
        axes[row, k].axis("off")

fig, axes = plt.subplots(4, 4, figsize=(10, 10))
show_row(TRAIN_GLIOMA, 0, "glioma")
show_row(TRAIN_MENINGIOMA, 1, "meningioma")
show_row(TRAIN_NOTUMOR, 2, "no_tumor")
show_row(TRAIN_PITUITARY, 3, "pituitary")
plt.tight_layout()
plt.show()
```




```
IMG_SIZE = (128, 128)
BATCH = 32

train_ds = keras.utils.image_dataset_from_directory(
    TRAIN_DIR, label_mode="categorical", color_mode="grayscale",
    image_size=IMG_SIZE, batch_size=BATCH,
    validation_split=0.2, subset="training", seed=42)

val_ds = keras.utils.image_dataset_from_directory(
    TRAIN_DIR, label_mode="categorical", color_mode="grayscale",
    image_size=IMG_SIZE, batch_size=BATCH,
    validation_split=0.2, subset="validation", seed=42)
```

```
test_ds = keras.utils.image_dataset_from_directory(
    TEST_DIR, label_mode="categorical", color_mode="grayscale",
    image_size=IMG_SIZE, batch_size=BATCH, shuffle=False)

class_names = train_ds.class_names
print(class_names)    # alphabetical: glioma_tumor, meningioma_tumor,
no_tumor, pituitary_tumor

train_ds = train_ds.cache().prefetch(tf.data.AUTOTUNE)
val_ds = val_ds.cache().prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.cache().prefetch(tf.data.AUTOTUNE)
```

```
Found 2870 files belonging to 4 classes.
Using 2296 files for training.
Found 2870 files belonging to 4 classes.
Using 574 files for validation.
Found 394 files belonging to 4 classes.
['glioma_tumor', 'meningioma_tumor', 'no_tumor', 'pituitary_tumor']
```

```
total = n_glioma + n_meningioma + n_notumor + n_pituitary
class_weight = {
    0: total / (4 * n_glioma),         # glioma_tumor
    1: total / (4 * n_meningioma),     # meningioma_tumor
    2: total / (4 * n_notumor),        # no_tumor (rarest, so the
largest weight)
    3: total / (4 * n_pituitary),      # pituitary_tumor
}
print("Class weights:", class_weight)

reg = keras.regularizers.l2(1e-4)

model = keras.Sequential([
    layers.Input(shape=(128, 128, 1)),
    layers.Rescaling(1.0 / 255),

    # augmentation (active only during training)
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.1),

    layers.Conv2D(32, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(64, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
```

```
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(128, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),

    layers.Conv2D(256, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.4),
    layers.Dense(4, activation="softmax"),
])

model.summary()
```

```
Class weights: {0: 0.8686440677966102, 1: 0.8728710462287105, 2:
1.8164556962025316, 3: 0.8675937122128174}
```

```
Model: "sequential_5"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ rescaling_1 (Rescaling)              │ (None, 128, 128, 1)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ random_flip_1 (RandomFlip)           │ (None, 128, 128, 1)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ random_rotation_1                    │ (None, 128, 128, 1)         │
0 │
│ (RandomRotation)                     │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ random_zoom_1 (RandomZoom)           │ (None, 128, 128, 1)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
```

```
──────┤
| random_contrast_1                | (None, 128, 128, 1)     |
0 |
| (RandomContrast)                 |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| conv2d_24 (Conv2D)               | (None, 128, 128, 32)    |
320 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| batch_normalization_4            | (None, 128, 128, 32)    |
128 |
| (BatchNormalization)             |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| max_pooling2d_12 (MaxPooling2D) | (None, 64, 64, 32)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout_4 (Dropout)              | (None, 64, 64, 32)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| conv2d_25 (Conv2D)               | (None, 64, 64, 64)      |
18,496 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| batch_normalization_5            | (None, 64, 64, 64)      |
256 |
| (BatchNormalization)             |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| max_pooling2d_13 (MaxPooling2D) | (None, 32, 32, 64)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout_5 (Dropout)              | (None, 32, 32, 64)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| conv2d_26 (Conv2D)               | (None, 32, 32, 128)     |
73,856 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| batch_normalization_6            | (None, 32, 32, 128)     |
512 |
```

```
| (BatchNormalization)                 |                             |
|
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| max_pooling2d_14 (MaxPooling2D) | (None, 16, 16, 128)     |
0 |
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dropout_6 (Dropout)                  | (None, 16, 16, 128)     |
0 |
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| conv2d_27 (Conv2D)                   | (None, 16, 16, 256)     |
295,168 |
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| batch_normalization_7                | (None, 16, 16, 256)     |
1,024 |
| (BatchNormalization)                 |                             |
|
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| max_pooling2d_15 (MaxPooling2D) | (None, 8, 8, 256)       |
0 |
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| global_average_pooling2d_1       | (None, 256)             |
0 |
| (GlobalAveragePooling2D)         |                             |
|
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dropout_7 (Dropout)                  | (None, 256)             |
0 |
├──────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dense_5 (Dense)                      | (None, 4)               |
1,028 |
└──────────────────────────────────────┴─────────────────────────────┴──────────
──────┘

 Total params: 390,788 (1.49 MB)

 Trainable params: 389,828 (1.49 MB)

 Non-trainable params: 960 (3.75 KB)
```

```
from tensorflow.keras.callbacks import ReduceLROnPlateau

model.compile(loss="categorical_crossentropy",
```

```
              optimizer=Adam(learning_rate=0.001),
              metrics=["accuracy"])

callbacks = [
    EarlyStopping(monitor="val_loss", patience=8,
restore_best_weights=True),
    ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3),
]

history = model.fit(train_ds, validation_data=val_ds, epochs=5,
                    class_weight=class_weight, callbacks=callbacks)

Epoch 1/5
72/72 ━━━━━━━━━━━━━━━━━━━━ 158s 2s/step - accuracy: 0.5941 - loss:
1.0846 - val_accuracy: 0.2979 - val_loss: 2.8133 - learning_rate:
0.0010
Epoch 2/5
72/72 ━━━━━━━━━━━━━━━━━━━━ 146s 2s/step - accuracy: 0.6442 - loss:
0.9342 - val_accuracy: 0.2979 - val_loss: 4.2325 - learning_rate:
0.0010
Epoch 3/5
72/72 ━━━━━━━━━━━━━━━━━━━━ 205s 2s/step - accuracy: 0.6847 - loss:
0.8101 - val_accuracy: 0.2979 - val_loss: 4.0044 - learning_rate:
0.0010
Epoch 4/5
72/72 ━━━━━━━━━━━━━━━━━━━━ 198s 2s/step - accuracy: 0.7043 - loss:
0.7718 - val_accuracy: 0.2753 - val_loss: 4.3026 - learning_rate:
0.0010
Epoch 5/5
72/72 ━━━━━━━━━━━━━━━━━━━━ 149s 2s/step - accuracy: 0.7456 - loss:
0.6649 - val_accuracy: 0.2648 - val_loss: 5.4984 - learning_rate:
5.0000e-04

test_loss, test_acc = model.evaluate(test_ds)
print("Test loss:", test_loss)
print("Test accuracy:", test_acc)

y_true = np.concatenate([y.numpy() for x, y in
test_ds]).argmax(axis=1)
y_pred = model.predict(test_ds).argmax(axis=1)

print(classification_report(y_true, y_pred, target_names=class_names))

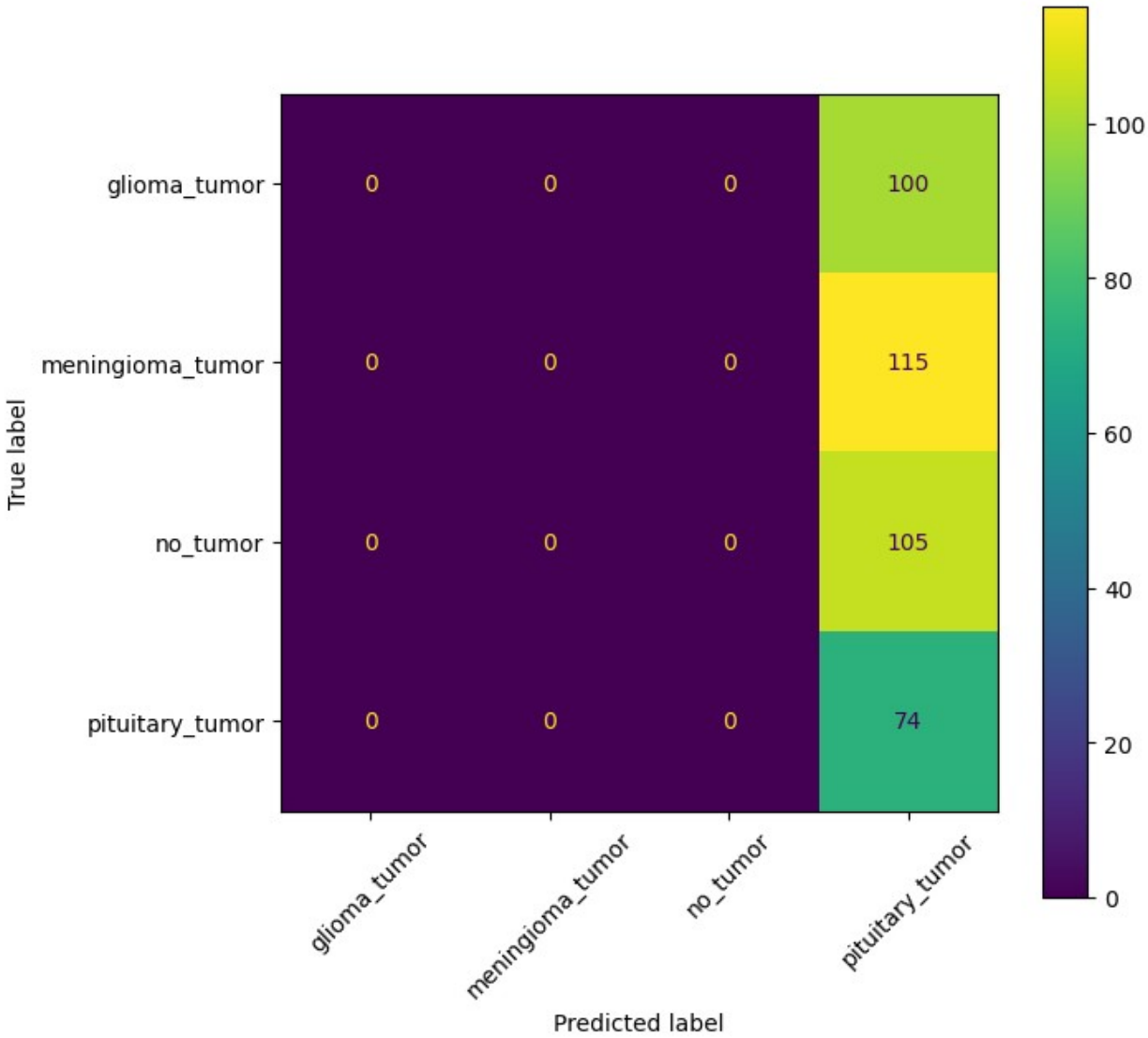
fig, ax = plt.subplots(figsize=(7, 7))
ConfusionMatrixDisplay.from_predictions(y_true, y_pred,
display_labels=class_names,
                                        xticks_rotation=45, ax=ax)
plt.show()
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 5s 356ms/step - accuracy: 0.1878 - loss:
3.9743
Test loss: 3.974263906478882
Test accuracy: 0.1878172606229782
13/13 ━━━━━━━━━━━━━━━━━━━━ 6s 425ms/step
                  precision    recall  f1-score   support

    glioma_tumor       0.00      0.00      0.00       100
meningioma_tumor       0.00      0.00      0.00       115
        no_tumor       0.00      0.00      0.00       105
 pituitary_tumor       0.19      1.00      0.32        74

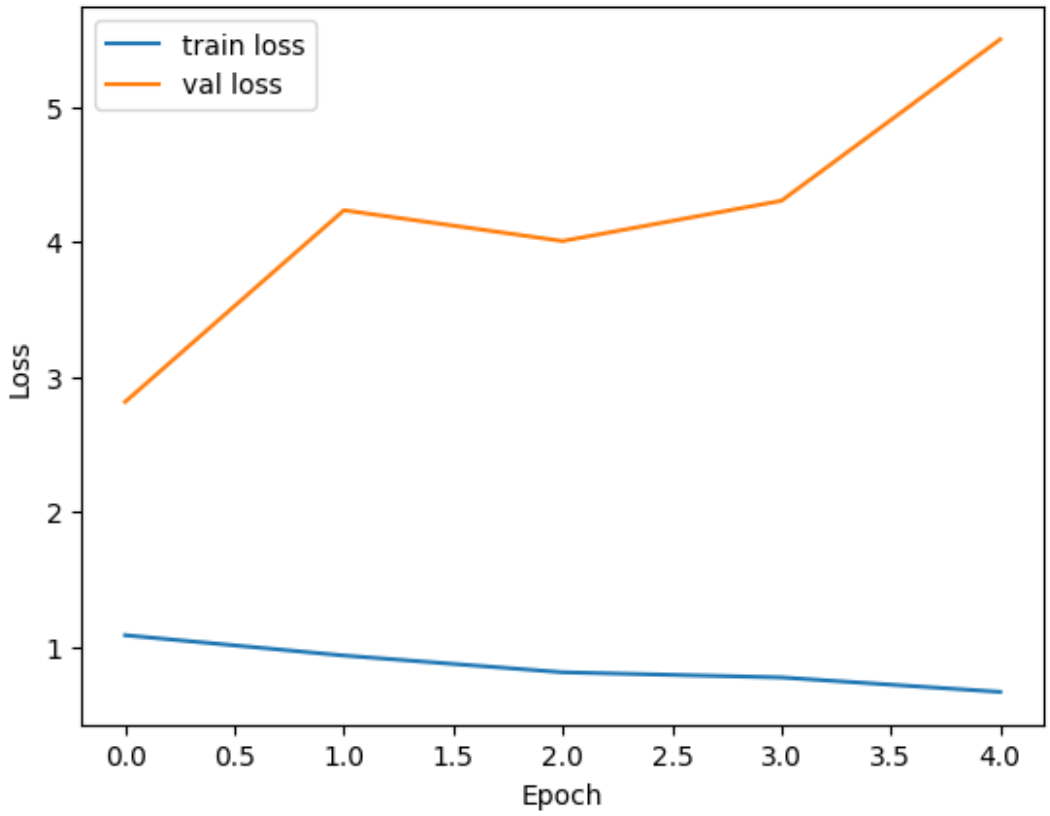
        accuracy                           0.19       394
       macro avg       0.05      0.25      0.08       394
    weighted avg       0.04      0.19      0.06       394

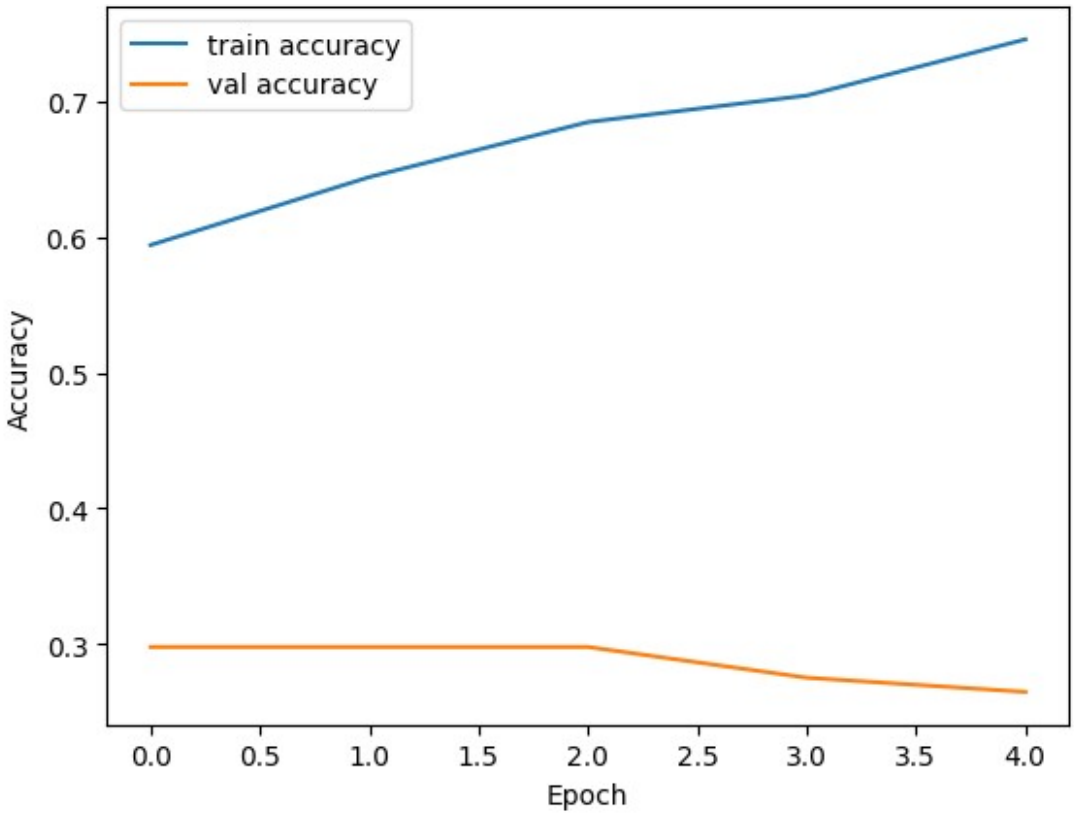

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificatio
n.py:1565: UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples. Use `zero_division`
parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```




```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


train loss
val loss
Loss
Epoch
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
1
2
3
4
5


train accuracy
val accuracy
Accuracy
Epoch
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
0.3
0.4
0.5
0.6
0.7